



NAME : HAMDAN ALI

SOFTWARE ENGINEERING INTERNSHIP — FINAL CAPSTONE

Week 8 — Day 40

**Presenting the Full-Stack Project, Explaining the Code Flow, and Submitting the
GitHub Repository**

Complete record — what I built, and every command I ran to actually publish it

Project: TaskFlow — Laravel Project And Task Management System

Submitted by: Hamdan Ali

BS Software Engineering — Air University, Islamabad

Repository: github.com/Hamdanali3/HindukushSoft-Technologies-Internship

1. What This Final Task Actually Involved

Day 40's brief was short but it's the one that ties everything else together: present the full-stack project, explain the code flow, and submit a GitHub repository. This document covers both halves of that honestly — the presentation package I built (README, architecture diagram, code-flow walkthrough, presentation script, Postman collection) and, just as importantly, the complete real command history of actually getting all eight weeks of internship work onto GitHub, including the mess I found along the way and how I cleaned it up.

I'm including the full command log deliberately, not just the polished end result, because getting a real project onto GitHub cleanly is its own skill — one that doesn't show up at all if I only present the finished repository link.

2. The Presentation Package

Before touching git, I built the actual materials someone would need to review and evaluate this capstone without me sitting next to them:

- README.md — the main project overview: features, tech stack, an embedded architecture diagram, database schema, API reference, setup instructions, and test results, plus an honest 'Known Limitations' section instead of overselling it.
- docs/CODE_FLOW.md — traces two real requests (one through the web UI, one through the API) file-by-file through every layer, built specifically to prove that the web and API controllers never duplicate an authorization rule — they both call the same `Project::hasAccess()` method.
- docs/PRESENTATION.md — a timed presentation script with a specific demo sequence and a prepared QAndA section, including a question about one of my own logged bugs.
- docs/architecture-diagram.svg — a real vector diagram showing the two entry points (web and API) converging into one shared validation/authorization/model layer.
- postman/TaskFlow.postman_collection.json — all 21 API endpoints, ready to import, since half the presentation demo relies on showing the API working through a real client.
- .gitignore and LICENSE — so the pushed repository is actually clean and properly licensed rather than an afterthought.

3. Discovering the Repository Was Already a Monorepo

My first assumption was that I'd be creating a brand-new repository just for TaskFlow. Running a plain git status from inside the taskflow project folder corrected that assumption immediately — I

was actually already inside a much larger git repository tracking my entire internship folder (Week1 through Week8), already connected to origin/main on a real GitHub repo, and it had pending, uncommitted Week 7 changes sitting there from earlier work.

```
git status
```

This mattered a lot for how I proceeded. If I'd blindly run `git add .` at that point, I would have mixed unrelated Week 7 edits, the new Week 8 documentation, and the actual Week 8 Laravel app into one confusing commit. Instead I decided to commit each piece separately and in order, so the commit history itself tells a clear story.

4. Committing Week 7 On Its Own First

Since Week 7's changes were already sitting there unrelated to anything I was doing today, I closed that out as its own clean commit before touching Week 8 at all:

```
cd "D:\internship ( HindukushSoft Technologies )"
git add Week7
git commit -m "Week 7: TaskFlow API and frontend - auth, error handling, and task management features"
git push
```

This pushed cleanly. Git did print a series of CRLF/LF line-ending warnings during the commit — that's just Git normalising line endings between how the files were saved on Windows versus how git internally stores them, not an error, and it doesn't affect the actual file contents.

5. Finding a Nested Git Repository Inside taskflow

With Week 7 out of the way, I checked what git saw inside the taskflow folder itself, expecting a normal list of files:

```
dir taskflow
```

Instead, when I ran `git status`, taskflow showed up as a single untracked item rather than a list of individual files. That's git's behaviour when a folder contains its own `.git` directory — it treats it as a nested repository ('gitlink') rather than walking inside it. I confirmed this directly:

```
dir taskflow\.git
```

Sure enough, taskflow had its own `.git` folder — almost certainly created automatically by Composer or Laravel's own installer at some point during setup. Since I wanted taskflow's files to merge into the one big internship repository as plain tracked files (not as a separate nested repo or submodule), I removed the inner `.git` folder entirely. This only deletes taskflow's own (empty, unpushed) git history — it does not touch any actual project files, code, or the database:

```
Remove-Item -Recurse -Force taskflow\.git
dir taskflow\.git # confirms path no longer exists
```

6. Copying the Day 40 Package Into the Real Project

With taskflow now just a normal folder, I copied the presentation package into it:

```
Copy-Item "Week8\Day40\.gitignore" "taskflow\.gitignore" -Force
Copy-Item "Week8\Day40\README.md" "taskflow\README.md" -Force
Copy-Item "Week8\Day40\LICENSE" "taskflow\LICENSE" -Force
Copy-Item "Week8\Day40\docs" "taskflow\docs" -Recurse -Force
Copy-Item "Week8\Day40\postman" "taskflow\postman" -Recurse -Force
```

Before staging anything, I deliberately checked what git would actually pick up, since taskflow is a full Laravel install with a vendor/ folder and a node_modules/ folder that could easily be hundreds of megabytes if the .gitignore didn't work correctly:

```
git status --untracked-files=all taskflow
```

The good news: vendor/ and node_modules/ correctly did not appear anywhere in that list, confirming the .gitignore was doing its job. The bad news: I spotted taskflow/docs/docs/... and taskflow/postman/postman/... in the list — a duplicated nested copy. This happened because the docs and postman copy commands had been run more than once, and since the destination folder already existed the second time, Copy-Item -Recurse nested the whole source folder inside it instead of merging.

```
Remove-Item -Recurse -Force taskflow\docs\docs
Remove-Item -Recurse -Force taskflow\postman\postman
dir taskflow\docs
dir taskflow\postman
```

After removing the duplicates, both folders showed exactly the right files — 4 in docs, 1 in postman.

7. Committing the Real TaskFlow Application

```
git add taskflow
git commit -m "Week 8: TaskFlow capstone - Laravel 12 project And task management system with full auth, REST API, and test suite"
```

This one commit captured 109 files and just over 16,500 lines — the entire working Laravel application: models, controllers, policies, form requests, API resources, migrations, Blade views,



routes, and the PHPUnit test suite, plus the newly added README, LICENSE, docs, and Postman collection.

8. Committing the Week 8 Planning And Documentation Folder

Separately from the actual application code, I committed the Week8/ folder itself — the Day 36 through 40 planning docs, per-day .docx write-ups, verification scripts, and reference copies of the code as it stood at the end of each day. I checked the file list first, the same way as before, specifically looking for anything unexpected:

```
git status --untracked-files=all Week8
```

This came back clean — no vendor/node_modules, no stray database dumps beyond the one small intentional SQLite file from the Day 36 verification step. So I committed and pushed everything gathered so far:

```
git add Week8
git commit -m "Week 8: Days 36-40 capstone documentation - planning, code flow, presentation, and GitHub submission guide"
git push
```

9. Catching Two Missing Documentation Files

While double-checking everything had actually made it in, I noticed the commit list for Week8 only showed Day38_Documentation.docx and Day39_Documentation.docx — Day 36 and Day 37's Word write-ups were nowhere in the list. I checked directly:

```
dir Week8\Day36\*.docx
dir Week8\Day37\*.docx
```

Both came back completely empty. These two files simply didn't exist in those folders on my machine — not a git problem, a genuinely missing deliverable. I regenerated both from scratch (matching the same structure and tone as the other day write-ups) and completed the submission:

```
git add Week8\Day36\Day36_Documentation.docx Week8\Day37\Day37_Documentation.docx
git commit -m "Week 8: add missing Day 36 and Day 37 documentation files"
git push
```



10. Full Command Reference

For my own record, every command run during this submission process, in order:

```
cd "D:\internship ( HindukushSoft Technologies )\taskflow"
dir
git status
cd "D:\internship ( HindukushSoft Technologies )"
git add Week7
git commit -m "Week 7: TaskFlow API and frontend - auth, error handling, and task management features"
git push
dir taskflow\.git
Remove-Item -Recurse -Force taskflow\.git
dir taskflow\.git
Copy-Item "Week8\Day40\.gitignore" "taskflow\.gitignore" -Force
Copy-Item "Week8\Day40\README.md" "taskflow\README.md" -Force
Copy-Item "Week8\Day40\LICENSE" "taskflow\LICENSE" -Force
Copy-Item "Week8\Day40\docs" "taskflow\docs" -Recurse -Force
Copy-Item "Week8\Day40\postman" "taskflow\postman" -Recurse -Force
git status --untracked-files=all taskflow
Remove-Item -Recurse -Force taskflow\docs\docs
Remove-Item -Recurse -Force taskflow\postman\postman
dir taskflow\docs
dir taskflow\postman
git add taskflow
git commit -m "Week 8: TaskFlow capstone - Laravel 12 project And task management system with full auth, REST API, and test suite"
git status --untracked-files=all Week8
git add Week8
git commit -m "Week 8: Days 36-40 capstone documentation - planning, code flow, presentation, and GitHub submission guide"
git push
dir Week8\Day36\*.docx
dir Week8\Day37\*.docx
git add Week8\Day36\Day36_Documentation.docx Week8\Day37\Day37_Documentation.docx
git commit -m "Week 8: add missing Day 36 and Day 37 documentation files"
git push
```

11. Final Commit History

Commit	Contents
Week 7: TaskFlow API and frontend...	All pending Week 7 backend/frontend work, closed out on its own
Week 8: TaskFlow capstone...	The real, working Laravel 12 application — 109 files, ~16,500 lines
Week 8: Days 36-40 capstone documentation...	Planning docs, per-day write-ups, verification scripts, presentation package
Week 8: add missing Day 36 and Day 37 documentation files	The two documentation files that were missing from the previous commit

12. What I'd Do Differently Next Time

- Run git status before copying anything, not after — I would have caught the monorepo situation and the nested .git repo before doing any file copying at all, rather than mid-way through.
- Copy-Item -Recurse into a folder that might already exist is dangerous — I'd check for the destination's existence first, or delete and recreate it, rather than assuming a second run merges cleanly.
- Verify every deliverable actually exists on disk before assuming a commit is complete — the missing Day 36/37 docs would have shipped silently if I hadn't cross-checked the commit's file list against what I expected to see.



13. Result

The repository at github.com/Hamdanali3/HindukushSoft-Technologies-Internship now contains: Week 7's complete backend/frontend work as its own clean commit, the fully working TaskFlow Laravel application (models, controllers, policies, API, tests, all passing), and the complete Day 36–40 documentation set — five Word write-ups, the planning docs, the verification scripts, and the full presentation package (README, architecture diagram, code-flow walkthrough, presentation script, Postman collection). Every command that got it there is recorded above, including the mistakes and how they were fixed, not just the final clean state.

THANK YOU!